

Orientação a objetos

Tratamento de exceções

Tratamento de exceções

- Além das exceções nativas do Java, é possível criar suas próprias exceções personalizadas.
- Para criar uma exceção personalizada, é necessário criar uma classe que estenda a classe `Exception` ou uma de suas subclasses.

Criando exceções

```
3 public class MinhaExcecao extends Exception {  
4     public MinhaExcecao() {  
5         super();  
6     }  
7  
8     public MinhaExcecao(String message) {  
9         super(message);  
10    }  
11 }
```

Criando exceções

- Na classe MinhaExcecao é uma exceção personalizada que estende a classe Exception.
- Ela possui dois construtores, um que não recebe parâmetros e outro que recebe uma mensagem de erro como parâmetro.
- Para lançar uma exceção personalizada em seu código, basta usar a palavra-chave throw seguida de uma instância da sua exceção personalizada.

Criando exceções

```
7 public static void main(String[] args) {
8     Scanner sc = new Scanner(System.in);
9     double n1, n2;
10    System.out.println("Digite dois números: ");
11    n1 = sc.nextDouble();
12    n2 = sc.nextDouble();
13    try {
14        divide(n1,n2);
15    } catch (MinhaExcecao e) {
16        System.out.println("Erro: "+e.getMessage());
17    }
18 }
19
20 private static void divide(double n1, double n2) throws MinhaExcecao {
21     if(n2==0) {
22         throw new MinhaExcecao("Impossível realizar a divisão por zero");
23     }
24     else {
25         System.out.println("O resultado da divisão é: "+(n1/n2));
26     }
27 }
28
29 }
```

```
7 public static void main(String[] args) {
8     Scanner sc = new Scanner(System.in);
9     double n1, n2;
10    System.out.println("Digite dois números: ");
11    n1 = sc.nextDouble();
12    n2 = sc.nextDouble();
13    try {
14        divide(n1,n2);
15    } catch (MinhaExcecao e) {
16        System.out.println("Erro: "+e.getMessage());
17    }
18 }
19
20 private static void divide(double n1, double n2) throws MinhaExcecao {
21     if(n2==0) {
22         throw new MinhaExcecao("Impossível realizar a divisão por zero");
23     }
24     else {
25         System.out.println("O resultado da divisão é: "+(n1/n2));
26     }
27 }
28
29 }
```

Desafio

- 1) Crie uma classe abstrata chamada Conta com as propriedades Saldo e Limite. Crie também métodos abstratos para Depositar e Sacar.
- 2) Faça implementação concreta desta classe com os métodos Depositar e sacar
- 3) Usando a classe anterior crie exceções caso o valor do saque seja negativo ou maior que o saldo disponível e se o valor do depósito seja negativo.

Desafio

- 1)

```
3 public abstract class Conta {
4     protected double saldo;
5     public abstract void depositar(double valor);
6     public abstract void sacar(double valor);
7     public Conta(double saldo) {
8         this.saldo = saldo;
9     }
10    public double getSaldo() {
11        return saldo;
12    }
13 }
```


Desafio

- 2)

```
3 public class ContaCorrente extends Conta {
4
5     public ContaCorrente(double saldo) {
6         super(saldo);
7     }
8
9     @Override
10    public void depositar(double valor) {
11        saldo += valor;
12    }
13
14    @Override
15    public void sacar(double valor) {
16        saldo -= valor;
17    }
18 }
19 }
```

Desafio

- 3)

```
3 public class SaldoInsuficienteException extends IllegalArgumentException {  
4     public SaldoInsuficienteException(String mensagem) {  
5         super(mensagem);  
6     }  
7 }
```

Desafio

• 3)

```
3 public abstract class Conta {
4     protected double saldo;
5     protected double limite;
6     public abstract void depositar(double valor) throws SaldoInsuficienteException;
7     public abstract void sacar(double valor) throws SaldoInsuficienteException;
8     public Conta(double saldo, double limite) {
9         this.saldo = saldo;
10        this.limite = limite;
11    }
12    public double getSaldo() {
13        return saldo;
14    }
15 }
```

Desafio

• 3)

```
3 public class ContaCorrente extends Conta {
4
5     public ContaCorrente(double saldo, double limite) {
6         super(saldo, limite);
7     }
8
9     @Override
10    public void depositar(double valor) throws SaldoInsuficienteException {
11        if (valor <= 0)
12            throw new SaldoInsuficienteException("Impossível depositar zero ou valor negativo");
13
14        saldo += valor;
15    }
16
17    @Override
18    public void sacar(double valor) throws SaldoInsuficienteException {
19        if (valor > saldo+limite)
20            throw new SaldoInsuficienteException("Saldo insuficiente");
21
22        else if (valor <= 0)
23            throw new SaldoInsuficienteException("Impossível sacar zero ou valor negativo");
24
25        saldo -= valor;
26    }
27 }
```

Desafio

• 3)

```
3 public class Principal {  
4     public static void main(String args[]) {  
5         ContaCorrente cc = new ContaCorrente(100,1000);  
6         System.out.println(cc.getSaldo());  
7         cc.depositar(10);  
8         cc.sacar(1500);  
9     }  
10 }
```

100.0

Exception in thread "main" DesafioAula11.SaldoInsuficienteException: Saldo insuficiente
at Excecoes/DesafioAula11.ContaCorrente.sacar(ContaCorrente.java:17)
at Excecoes/DesafioAula11.Principal.main(Principal.java:8)

Desafio

• 3)

```
3 public class Principal {  
4     public static void main(String args[]) {  
5         ContaCorrente cc = new ContaCorrente(100,1000);  
6         System.out.println(cc.getSaldo());  
7         cc.depositar(10);  
8         cc.sacar(-100);  
9     }  
10 }
```

100.0

Exception in thread "main" DesafioAula11.SaldoInsuficienteException: **Impossível sacar zero ou valor negativo**
at Excecoes/DesafioAula11.ContaCorrente.sacar(ContaCorrente.java:20)
at Excecoes/DesafioAula11.Principal.main(Principal.java:8)

Desafio

- 4) Crie uma classe chamada Animal com os atributos privados Nome e Idade. Também deve possuir os métodos get e set para encapsulamento. Crie também um método abstrato chamado EmitirSom. Crie duas classes que herdam de Animal, uma chamada Cachorro e outra chamada Gato, que implementam o método EmitirSom.
- 5) Faça as implementações dos métodos do exercício 1 para uma ContaInvestimento e ContaPoupanca e apresente mensagens na tela. A ContaInvestimento deve possuir um método atualizaRendimentos que deve acrescer 5% ao saldo. Já a ContaPoupanca deve possuir um método atualizaJuros que deve acrescer 3% ao saldo da conta.
- 6) Crie uma classe abstrata chamada Veiculo com as propriedades Marca, Modelo e Ano. Crie dois métodos abstratos chamados Acelerar e Frear. Crie duas classes que herdam de Veiculo, uma chamada Carro e outra chamada Moto, que implementam os métodos Acelerar e Frear.
- 7) Crie uma classe abstrata chamada Pessoa com as propriedades Nome e Cidade. Crie também um método abstrato chamado EfetuarCompra. Crie duas classes que herdam de Pessoa, uma chamada PessoaFisica e outra chamada PessoaJuridica, que implementam o método EfetuarCompra. Crie atributos específicos para cada uma das novas classes, por exemplo, CPF em PessoaFisica e CNPJ em PessoaJuridica.
- 8) Crie uma classe abstrata chamada Funcionario com as propriedades Nome e Salário. Crie também um método abstrato chamado CalcularSalario. Crie duas classes que herdam de Funcionario, uma chamada Gerente e outra chamada Vendedor, que implementam o método CalcularSalario. O gerente deve ganhar um bônus de 20%.
- 9) Analisando os exercícios anteriores, quais implementações podem ser desenvolvidas com interface?

Desafio

- 10) Crie uma interface chamada Produto que contenha os seguintes métodos: `getNome()`, `getPreco()` e `getDescricao()`.
- 11) Crie as classes Livro e CD que implementem a interface Produto. Implemente os métodos da interface para que eles retornem as informações correspondentes ao nome, preço e descrição do produto.
- 12) Crie uma interface chamada AcessoDados que contenha os seguintes métodos: `conectar()`, `desconectar()`, `inserir()`, `atualizar()` e `excluir()`.
- 13) Crie uma classe BancoDeDados que implemente a interface AcessoDados. Implemente os métodos da interface para que eles realizem as operações de conectar-se ao banco de dados, desconectar-se, inserir dados, atualizar dados e excluir dados.
- 14) Usando a Pessoa crie um construtor que inicialize esses atributos e um método `validarIdade` que lance uma exceção caso a idade informada seja menor que zero ou maior que 120.
- 15) Crie uma classe Data com três atributos: dia, mes e ano. Crie um construtor que inicialize esses atributos e um método `validarData` que lance uma exceção `IllegalArgumentException` caso a data informada não seja válida (por exemplo, dia 31 de fevereiro).
- 16) Crie uma classe Triangulo com três atributos: lado1, lado2 e lado3. Crie um construtor que inicialize esses atributos e um método `validarTriangulo` que lance uma exceção `IllegalArgumentException` caso os valores informados não formem um triângulo válido (por exemplo, se a soma de dois lados for menor ou igual ao terceiro lado).
- 17) Crie uma classe Agenda com um método `adicionarContato` que receba o nome e o telefone de um contato e lance uma exceção `IllegalArgumentException` caso o nome ou o telefone sejam nulos ou vazios.